



HiveRelay

BLINDSPARK · PEAR ECOSYSTEM CORE

v0.17.0 → v0.24.1 · release digest

The Custody, Notify & **Peerit** Release Cycle

Everything that shipped since our last announcement at **0.17** — the network upgrades, the launch of the app-facing service layer, the design of **Peerit**'s signed-outbox model, and how the Pear browser now talks to a federation of blind relays.

8 minor releases

Blind shard custody — k-of-n dispersal

Proof-of-Storage Tier 1 & 2

notify + **outboxlog** services

DHT-over-WS browser P2P

5 HIGH audit blockers closed

00 The short version

Between **0.17** and **0.24.1**, HiveRelay grew from an app-seeding appliance into a **content-blind service substrate**. Three storylines run through the cycle:

STORYLINE 1

Network & custody

Relays learned to **prove** what they store, **disperse** secrets so no single operator can reconstruct them, discover each other over the DHT, and pipe raw P2P bytes — all while never reading a byte of content.

STORYLINE 2

The service layer

A headless **Services Layer** apps consume over RPC: encrypted wake-up push (**notify**), a signed single-writer outbox (**outboxlog**), verifiable randomness, and the blind shard store — the plumbing that makes **Peerit** possible.

STORYLINE 3

Peerit & the browser

The Pear browser moved to a **per-relay subscription** model with provenance badges, DHT-over-WebSocket for real-time P2P, and a signed catalog it can verify — no central index, no forced federation.

RELEASE MAP AT A GLANCE

VERSION	DATE	HEADLINE	THEME
0.17.0	06-16	Seed & list your own app from the appliance dashboard; app icons in the catalogue	Operator UX
0.18.0	06-16	Paid pin-lease (zero-custody) + replicable signed catalog bee	Incentives
0.18.1	06-16	Services tab (host an AI model) + Tier-2 index layer sidecar	Discovery
0.19.x	06-17→22	DHT-resolvable relay discovery, poker services, honest storage accounting	Network
0.20.x	06-22→24	Proof-of-Storage (Tier 1 & 2); Pear/Bare client importability; honest "served" metric	Verifiability
0.21.x	07-02	notify + outboxlog + blind shard-store + witnesslog/repairticket	Service layer
0.22.0	07-03	Production-hardened dht-relay-ws pure-pipe transport	Transport
0.23.0	07-03	Operator storage designation (give HiveRelay N GB, live)	Operator
0.24.0	07-06	Blind-custody dispersal wired end-to-end; 5 HIGH audit blockers closed	Security
0.24.1	07-07	Outbox namespace env registration (the Peerit provisioning fix); release automation	Peerit ops

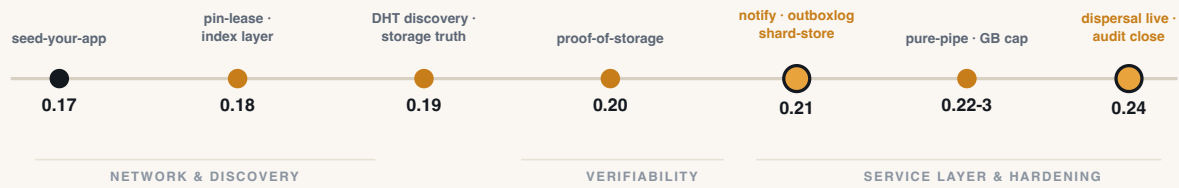


Fig 1 — Eight releases, three phases. The larger amber nodes (0.21, 0.24) are the load-bearing ones for Peerit.

How the pieces stack up

HiveRelay is a two-layer system. Apps never touch Hypercore directly — they consume headless **services** over RPC, and those services sit on a relay **kernel** that stays blind to content.

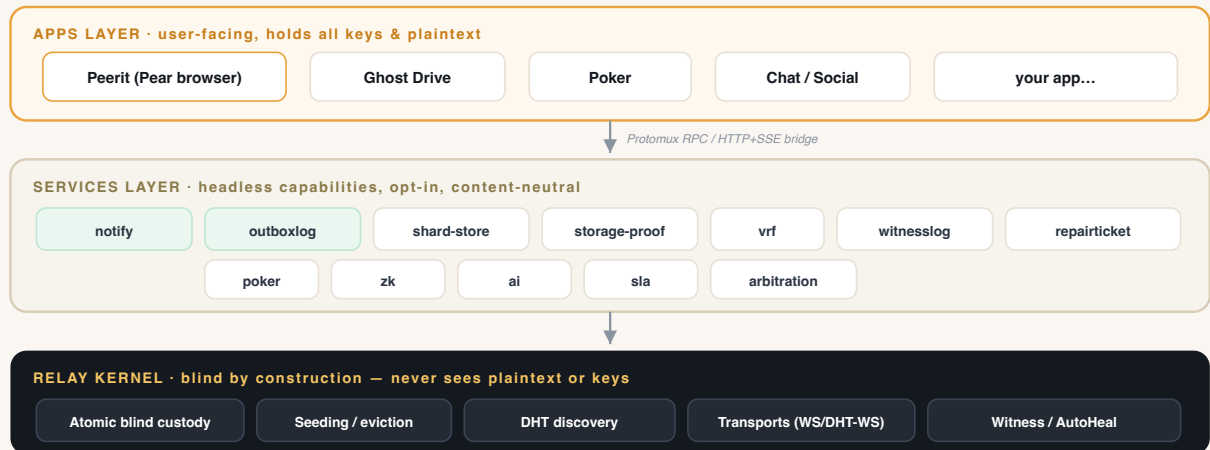


Fig 2 — The two-layer architecture. Green services (notify, outboxlog) are what Peerit consumes; everything below the RPC line is content-blind.

01 Network upgrades

The infrastructure changes: discovery, verifiability, custody, transport, and storage honesty.

DHT-RESOLVABLE DISCOVERY

0.19.0

A relay publishes a **signed self-description** — `pubkey → { gatewayUrl, indexRoom }` — as a `hyperdht` mutable record keyed by its identity.

Anyone who knows only a relay's pubkey can resolve its current gateway over the DHT, signature-verified, with **no trusted directory** and no Mainline/pkarr dependency. Republishes every 30 min. Carries only already-public data (blind-safe).

TIER-2 INDEX LAYER

0.18.1

Catalogue, pins, relay-directory and verifications mirror into a signed, **JMESPath-queryable** `schema-sheets` room clients blind-replicate read-only. Runs as a **dependency-isolated sidecar** (corestore-7/hypercore-11) that bridges until the relay's own hypercore-11 migration lands. Off by default — no sidecar, no `indexRoom`, clients fall back to `catalogBeeKey / /catalog.json`.

HONEST STORAGE ACCOUNTING

0.19.3–.4 ·
0.20.2

Root-caused the fleet **disk-full incidents**: accounting read ~0 on registry-driven relays, so the adoption cap never bound and disks filled to 100%. Now measures the **real on-disk corestore footprint** (allocated blocks, matching `du`), and all four adoption guards gate on it. Paired with an honest **"Data served"** counter that sums bytes across every core, not just `seedCore` traffic.

OPERATOR STORAGE DESIGNATION

0.23.0

"Give HiveRelay **N GB**" — live, no restart, from the Blindspark page or `POST /api/manage/config`. The relay re-caps the adoption gate, enables eviction, and sheds surplus blind fragments down to the cap. Eviction now triggers on **measured usage vs the cap**, not just whole-disk pressure — so it shrinks to fit even on a near-empty physical disk. Replication floor + margin still protect the network.

PURE-PIPE TRANSPORT HARDENING

0.22.0

Production-hardens `dht-relay-ws` for operators running a content-blind DHT byte-pipe 24/7. Adds a connection supervisor (liveness, first-frame deadline, backpressure termination), **byte metering** for billing, and a vendored+patched `@hyperswarm/dht-relay` fixing three confirmed prod blockers. Every bound is **content-neutral** (frame lengths, timings — never the Noise-encrypted payload), preserving the §512(a) transitory-conduit posture.

PAID PIN-LEASE + CATALOG BEE

0.18.0

Zero-custody paid seeding: a relay can charge a publisher to keep their drive seeded for a window; funds settle to the operator's **own Lightning node** and the only durable artifact is a lease deadline. Self-host stays free. Pricing keys on `(appKey, maxStorage, leaseDays)` — never content. Plus a **replicable signed catalog bee** so consumers verify a catalog over P2P instead of polling HTTP.

SPOTLIGHT — PROOF-OF-STORAGE: TRUSTLESS SEED VERIFICATION

0.20.0

Tier 1 — `verifySeeded` . Confirm a relay actually holds & serves a specific app without trusting its self-reported catalog. Downloads both cores to completion; Hypercore verifies every block against the drive's signed Merkle root on arrival — a relay **cannot fake content it doesn't hold**.

Tier 2 — `proveSeeded` . A signed challenge-response: the caller samples random block indices; the relay reads each from **local storage only** and returns a Hypercore Merkle proof signed with its identity key. Verified against the drive key alone + relay signature (attribution, nonce-freshness, no replay).

Hardened against adversarial review: a **privacy gate** (blind drives return `NOT_SEEDED` , so `prove` can't become a possession oracle), a sybil-resistant **global** proof-work rate cap, and a phantom-core DoS guard (only `appRegistry` keys — never `store.get` on attacker input).

01b Blind shard custody

The cycle's deepest network primitive — landed in 0.21.0, wired end-to-end and proven in 0.24.0.

The **blind shard store** (`shard-store`) is a content-addressed blob surface for custody shares. You `PUT` opaque ciphertext and get back `shard:<hash>` where `hash = blake2b-256(ciphertext)`; a `GET` returns the exact bytes and the caller re-hashes to verify. **The relay is trusted for nothing and never introspects a blob.** Every `PUT` is authorized by a signed **custody pin** — the hash is bound to a `shareIndex` assigned to this specific relay in a verified custody-intent's signed `shareManifest`, so a relay can only pin the exact share it was assigned.

THE PATTERN — "PUBLIC PLAINTEXT, BLIND CUSTODY"

Encrypt content with a random key, store the ciphertext, and **disperse the KEY as k-of-n blind shards** across independent relays. No single operator — and no `k-1` colluding operators — can produce the plaintext. Any `k` let the reader reconstruct at their own edge. Built on PVSS secret-sharing married to the content-addressed store.

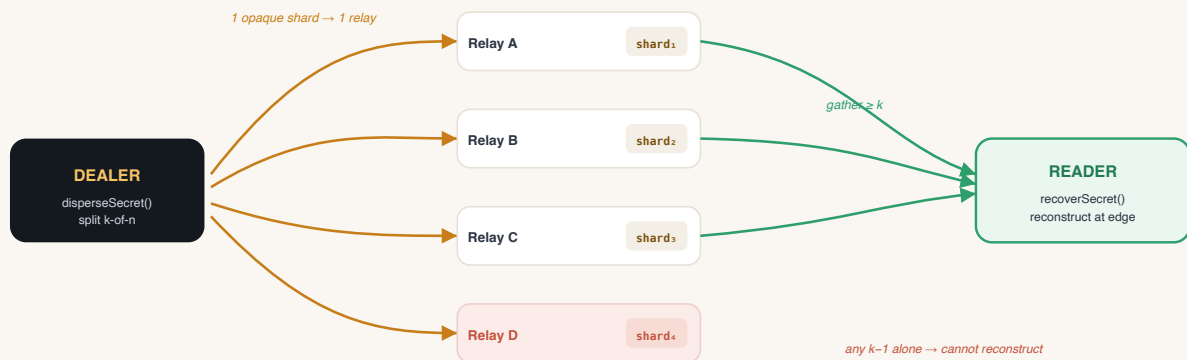


Fig 3 — k-of-n blind custody dispersal (here 3-of-4). Proven over live relays and against four full RelayNode instances on a testnet DHT.

WHAT 0.24.0 DELIVERED

- The `shard-store` HTTP adapter, shipped-but-unmounted in 0.21, is now **wired into the relay request path** — `/api/v1/shard` is reachable.
- **Relay-backed custody authorization:** a `PUT` is accepted only when the pin names a custody intent this relay indexed and its own `relayPubkey → shareIndex` binding matches.
- One-call app API: `disperse()` / `recover()` — signing & intent-publishing injected so the pin stays byte-identical to the relay's verifier.
- End-to-end proofs across 3–5 live relays **and** four full nodes on a testnet DHT.

HONEST LIMITS (WHAT'S NOT YET PROVEN)

- The fleet-scale run against production relay URLs has **not** been executed yet.
- Dispersing across relays a **single operator controls** proves the mechanism, not the security property — that needs **independent operators** (Gate 2, pending).
- A Bare/client-only dealer still awaits the client custody signer's `shareManifest` v2 mirror (added in 0.24.0; wiring tracked separately).

AVAILABILITY PRIMITIVES — WITNESSLOG & REPAIRTICKET 0.21.0

Two OutboxLog-backed signed logs complete the placement/recovery story. `witnesslog` records signed third-party availability observations (independent, non-storage witnesses sign what they see); `repairticket` records signed

repair requests, claims, receipts and closures for the self-healing AutoHeal loop. Both HTTP-bridge at `/api/witness` and `/api/repair` and inherit the OutboxLog engine's hardening.

02 The launch of Peerit

0.21 shipped the two services Peerit runs on: encrypted wake-up push and a signed single-writer outbox.

Peerit is the Pear-runtime browser/app in the ecosystem. It keeps all keys and plaintext on the user's device and leans on HiveRelay for exactly two things a pure-P2P app can't do alone: **waking a sleeping device**, and a **relay-assisted append/sync path** for its outbox. Both landed as opt-in services in 0.21, and 0.24.1 fixed the last provisioning gap that stopped a fresh appliance from accepting Peerit's writes.

NOTIFY — ENCRYPTED WAKE-UP PUSH

0.21.0

An always-on relay wakes a peer's app through **APNs / FCM / WebPush / runtime** without ever seeing **who** is being notified or **why**. Capabilities split into a **ReceiveCap** (the device consents to be woken) and a **SendCap** (a peer is authorized to wake it); the relay verifies domain-separated Ed25519 signatures and stays blind — payloads are opaque ciphertext, wakes are generic.

Modes: direct · watch (Mode 2) · presence-fallback. Delivery events are relay-signed and redacted — no provider tokens, no plaintext, ever leak.

OUTBOXLOG — SIGNED SINGLE-WRITER LOG

0.21.0

A **Peerit-compatible**, blind-sealed, signature-gated per-pubkey append log with a token-gated HTTP/SSE bridge, a hypercore operation journal, and an in-process swarm hub for browser peers. **The relay stores and serves signed rows it cannot read.** Only the configured writer can append; entry bodies stay opaque.

It **re-verifies signatures on load** and drops unverifiable rows — the persisted state file is no longer a trust root; only the writer key is.

THE GOVERNING RULE

"Push wakes the app; P2P sync gives the app truth."

notify only nudges a device to reconnect. The device then syncs authoritative state from the app's own Hypercore/Autobase/direct-P2P model — the relay is never a mailbox and never the source of truth.

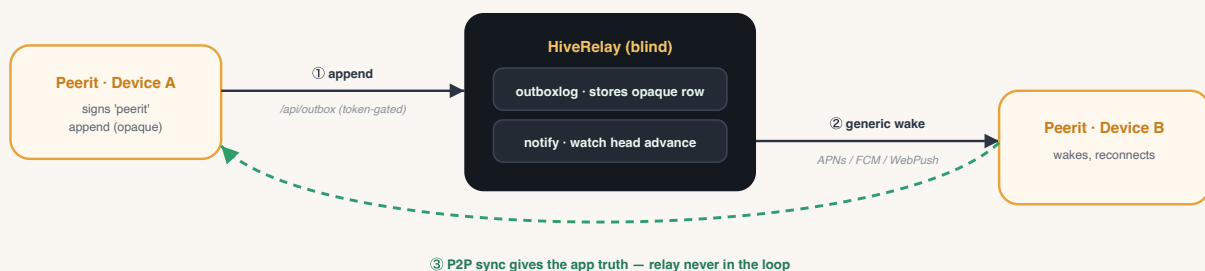


Fig 4 — Peerit's write → wake → sync loop. The relay stores an opaque row and fires a content-free wake; real state moves device-to-device.

Also in 0.21: a **Mode-2 watch runtime** — a **notify-feed-head** watch composes with a co-resident **outboxlog**, so when the watched outbox head advances the device gets one opaque, coalesced wake. And **0.24.0** added an operator **takedown surface** (`/api/admin/takedown` · `/restore` · `/takedowns`) gated by a dedicated admin credential distinct from the browser sync token — opaque (`appId`, `key`) only, content never read, safe-by-default (no credential ⇒ **404**).

02b Peerit's design, in depth

Namespaces, blind sealing, and the provisioning footgun 0.24.1 closed.

NAMESPACE-SIGNED APPENDS

The outbox is deliberately **app-neutral**. Every signed record carries a **namespace**, and each app signs under its own — **Peerit signs 'peerit'**. The namespace isn't a loose label: it's bound **into the signature** through the message prefix (`pear.app.<driveKey>:<ns>:...`), so a signature can't be replayed across namespaces — cross-namespace reuse simply fails verification. (An audit flagged "namespace not in signed payload" as HIGH; it was a false positive for exactly this reason.)

BLIND SEALING & DURABILITY

Rows are **blind-sealed** — the relay stores ciphertext it can't read. Persistence treats the **hypercore journal as the per-append WAL** and the snapshot as a periodic checkpoint; restore = checkpoint + journal-tail replay. SSE honors backpressure (drops live events under a full socket buffer, resumes on drain). The swarm hub — for browser peers with no UDP — gained explicit `destroy()` / `close()`.

THE SIGNED RECORD

```
// signature covers namespace via
prefix
prefix: pear.app.<driveKey>:peerit:...
writer: <ed25519 pubkey>
seq:    monotonic ↑
body:   <opaque ciphertext>
sig:    ed25519(prefix||body)
```

Relay verifies **sig · writer · seq · namespace-registered**, then stores. It never learns what **body** means.

THE PROVISIONING FOOTGUN — AND THE 0.24.1 FIX

Because the outbox is app-neutral, a relay **rejects any record whose namespace isn't registered** with **unknown namespace** (HTTP `400`). Enabling `HIVERELAY_OUTBOXLOG=1` alone was not enough — an ENV-provisioned fleet/appliance box `400` 'd **every** Peerit publish because there was no env-only way to admit the **'peerit'** namespace. **This was the single most common reason a fresh box refused an otherwise-working app.**

0.24.1 adds `HIVERELAY_OUTBOXLOG_NAMESPACE=peerit` (mapping to `config.outboxlog.namespace`), handed to the engine's namespace registry so a **'peerit'**-signed append is accepted. It's a **default, not an override** — a namespace in `config.json` wins, and env-unset leaves behavior unchanged. Wired via the pure, unit-tested `applyOutboxlogNamespaceEnv` helper.

TWO WAYS TO REGISTER A NAMESPACE

PATH	HOW	PRECEDENCE
Node / config	<code>config.outboxlog.namespace = "peerit"</code>	Wins over env (persisted)
Config (multi)	<code>config.outboxlog.namespaces = { ... }</code>	Register several at once
Bare / appliance	<code>HIVERELAY_OUTBOXLOG_NAMESPACE=peerit</code>	Default only (0.24.1+)

The browser-side **key-at-rest + CSP hardening** for Peerit lives in the Peerit repo, out of scope for the relay release — noted in the 0.24.0 supply-chain section.

03 Pear browser integrations

How the browser discovers apps across a federation of relays it doesn't have to trust.

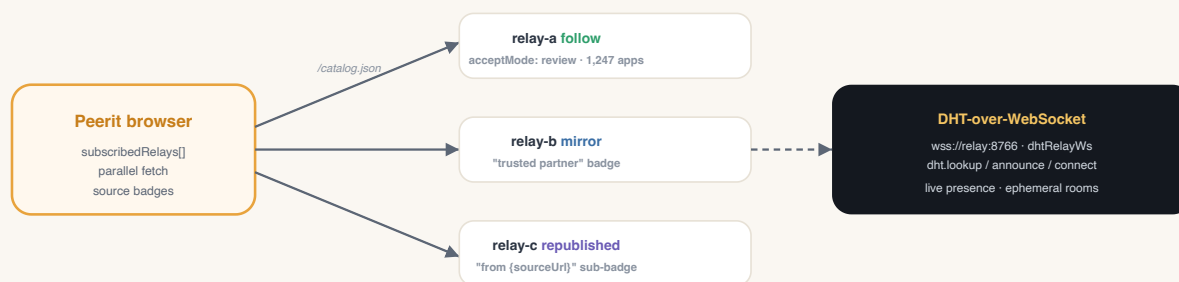
The big shift this cycle: away from **one global merged catalog** and toward **per-relay catalogs the user explicitly subscribes to**. Operators manually approve apps; the browser queries each relay separately and shows **provenance** instead of silently merging. Federation became **metadata for the UI** (who a relay follows / mirrors / republishes) rather than a discovery mechanism.

OLD MODEL

- One global merged catalog
- Auto-sync propagated apps without operator consent
- No way for browsers to join HyperDHT lookups (no UDP)

NEW MODEL

- Each relay has a **local catalog**; operators approve apps
- Federation is **per-subscription** & user-driven — provenance only
- Browsers run a **real HyperDHT** by tunneling through **dhtRelayWs**



Discovery is user-driven: subscriptions never auto-discover from peers, and republished apps always attribute back to the source.

Fig 5 — Per-relay subscriptions with provenance badges, plus an optional real HyperDHT tunneled through any relay that opts into dhtRelayWs.

WHAT THE BROWSER GAINED ACCESS TO

- **Per-relay catalog fetch** — each `/catalog.json` returns **apps**, **federation**, **acceptMode**; every row tagged with its source relay.
- **Signed catalog bee** (`catalogBeeKey`, 0.19.1) — clients that can replicate + verify a signed P2P catalog **prefer it** over the HTTP firehose.
- **DHT-over-WS** (0.22.0) — a real HyperDHT in the browser for live presence, ephemeral chat, and rooms that don't fit the seeded-app model.
- **Quick-add flow** — one-tap "subscribe to this relay" from a shared link or QR; never auto-trusts.

READ-ONLY BACKEND TOUCHPOINTS

<code>/catalog.json</code>	apps + acceptMode + federation
<code>/status</code>	capability discovery (transports, services)
<code>wss://...:8766</code>	DHT-over-WS (when dhtRelayWs on)
<code>wss://...:8765</code>	Hypercore replication over WS

All four are no-auth from the browser's perspective. Operator `/api/manage/*` endpoints are out of scope for the browser.

ENABLING FOUNDATIONS FROM 0.17 & 0.20

0.17 gave operators the "Seed an app" dashboard button and a copy-to-clipboard **PearBrowser subscribe URL** — apps auto-list in `/catalog.json`, which the browser reads. **0.20.1** fixed **Pear/Bare importability** of the client

(removed a Node-only `os` import) so PearBrowser worker bundles can load `p2p-hiverelay-client` directly — a hard prerequisite for the embedded P2P path.

04 Security hardening & operator notes

0.24.0 closed five HIGH audit blockers before public exposure. Here's what changed and what operators must do.

HIGH · CLOSED

FORK-PROOF QUARANTINE

Any followed relay could previously quarantine any drive network-wide (unauthenticated censorship/DoS). Now `verifyForkEvidence` requires **two conflicting signed heads** verified against the drive key, and pulled proofs are gated by a **trusted-observer allow-list** (empty = fail-closed).

HIGH · CLOSED

SHARD BYTE INTEGRITY

Shard bytes are deleted only when **both** the pin-registry and dedup ref-counts reach zero, under a per-hash lock — a concurrent dedup-PUT racing a sweep can no longer delete live bytes, and a duplicate signed PUT no longer orphans bytes. Adds `evictUnderPressure` honoring the `retainUntil` floor.

HIGH · CLOSED

SIGNED SUPPLY CHAIN

The fleet updater **verifies the target tag is signed** by a trusted key before checkout (fail-closed, documented break-glass), the Umbrel compose is pinned by `@sha256` digest, and image publishing gains **cosign keyless signing** + a verify step.

OTHER SECURITY FIXES ACROSS THE CYCLE

- **0.21.0** `notify.delivery-event` authenticated — closes a cross-tenant metadata IDOR; status counts scoped to caller.
- **0.21.1** `shard-store` HTTP adapter rate-limits per IP (before it was mounted).
- **0.21.1** SSE keepalive pings honor backpressure/teardown (witnesslog, repairticket).
- **0.22.0** `trustProxy` /XFF rate-limit keying — fixed a global-bucket collapse behind a TLS proxy (live on two relays).
- **0.21.1** Custody-intent signing regression from 0.21.0 fixed — **do not deploy 0.21.0 for custody**.

OPERATOR ACTION ITEMS

- **Running Peerit?** Register the namespace: `HIVERELAY_OUTBOXLOG_NAMESPACE=peerit` (or `config.outboxlog.namespace`). Without it, every publish `400` s.
- **Dispersal?** Turn on the `shard-store` service (Services tab / `plugins`) — mounting only makes the surface reachable.
- **Takedown surface?** Provision a dedicated `outboxlog.adminKey` distinct from the sync token; absent ⇒ routes stay `404`.
- **Supply chain:** the signing key, allowed-signers file, and cosign identity are a one-time provisioning — see `docs/SUPPLY-CHAIN.md`.
- **Storage:** set a GB cap from Blindspark (0.23.0) — live, no restart.

0.24.1 — RELEASE AUTOMATION

The **community Umbrel store now auto-syncs on every release** (prereleases included), so it never lags the fleet; the built multi-arch image digest flows into the store's `docker-compose.yml` `@sha256` pin on every sync. The workspace/docs alignment audit became **advisory** (a `::warning::` annotation) instead of blocking the release — while `npm audit`, lint, the secrets scanner, ecosystem-consumers, and unit tests stay hard-blocking.

HiveRelay 0.24.1 · p2p-hiverelay · p2p-hiveservices ·
p2p-hiverelay-client · p2p-hiverelay-verifier
(lockstep)



Compiled from CHANGELOG.md (0.17.0–0.24.1), docs/SERVICES.md, and the PearBrowser integration briefs. Diagrams are illustrative. Where the changelog marks a capability unproven-at-fleet-scale or pending an independent-operator gate, this document preserves that caveat.